Análise Comparativa de Algoritmos de Ordenação em Estruturas de Dados Dinâmicas e Estáticas

Elder Rayan Oliveira Silva, Samuel Wagner Tiburi Silveira, Diogo Gomes Figueiredo,
Manoel Junio Duarte da Silva, Pedro Yan Alcantara Palácio, Antonio Pereira da Luz Neto
Universidade Federal do Cariri – UFCA
Engenharia de Software
Disciplina: Estrutura de Dados

***Resumo*—Este trabalho apresenta uma análise comparativa do desempenho de cinco algoritmos clássicos de ordenação — Bubble Sort, Selection Sort, Insertion Sort, Quick Sort e Merge Sort — aplicados a três estruturas de dados distintas: vetor convencional, lista duplamente encadeada dinâmica e lista duplamente encadeada estática. A aplicação real escolhida foi um sistema de ranking de alunos, no qual registros contendo identificador e nota são ordenados de forma crescente. Os experimentos foram conduzidos na linguagem C, com 100 execuções por cenário, variando o volume de dados (100, 1.000 e 10.000 elementos) e o tipo de entrada (aleatório, ordenado e inversamente ordenado). Os resultados confirmam a superioridade assintótica dos algoritmos $O(n \log n)$: o Merge Sort apresentou os menores tempos médios em praticamente todos os cenários, com tempo de 1,36 ms para 10.000 elementos aleatórios em vetor, enquanto o Bubble Sort registrou 142,05 ms na mesma configuração. A análise do impacto da estrutura de dados revelou que o vetor convencional oferece o melhor desempenho geral, seguido pela lista dinâmica, sendo a lista estática a mais lenta devido ao custo de navegação por índices. O Merge Sort foi selecionado como algoritmo ideal para a aplicação proposta, dada sua garantia de desempenho $O(n \log n)$ em todos os casos e sua propriedade de estabilidade.**

***Index Terms*—Algoritmos de ordenação, estruturas de dados, lista encadeada, análise de desempenho, complexidade computacional.**

I. Introdução

A escolha adequada de algoritmos de ordenação e estruturas de dados é um problema fundamental em Ciência da Computação, com impacto direto no desempenho de sistemas reais [1], [2]. Em aplicações que manipulam grandes volumes de dados — como sistemas de ranking acadêmico, plataformas de *e-commerce* e aplicativos de entrega — a eficiência da ordenação pode determinar a viabilidade do sistema sob restrições de tempo de resposta.

Embora a análise teórica de complexidade assintótica forneça uma estimativa do comportamento dos algoritmos, fatores práticos como localidade de cache, overhead de alocação de memória e a estrutura de dados subjacente influenciam significativamente o desempenho real [3], [4]. Assim, a avaliação empírica complementa a análise teórica e permite decisões de projeto mais informadas.

Este trabalho tem como objetivo analisar o impacto da escolha de estruturas de dados e algoritmos de ordenação no desempenho de uma aplicação real de ranking de alunos. Para isso, foram implementados cinco algoritmos clássicos de ordenação (Bubble Sort, Selection Sort, Insertion Sort, Quick Sort e Merge Sort) sobre três estruturas de dados (vetor, lista duplamente encadeada dinâmica e lista duplamente encadeada estática), com avaliação empírica em diferentes volumes e padrões de entrada.

II. Fundamentação Teórica

A. Estruturas de Dados Utilizadas

Para avaliar o impacto da organização da memória no desempenho dos algoritmos, foram implementadas três estruturas de dados para armazenar registros de alunos (identificador e nota):

1) Vetor Convencional: Estrutura contígua em memória com acesso direto por índice em $O(1)$. Oferece a melhor localidade de cache, mas possui tamanho fixo.

2) Lista Duplamente Encadeada Dinâmica: Utiliza alocação de memória sob demanda via `malloc()` e `free()`. Cada nó armazena identificador, nota e ponteiros para o nó anterior e o próximo. Oferece flexibilidade de tamanho, mas sofre com fragmentação de memória e localidade de cache prejudicada.

3) Lista Duplamente Encadeada Estática: Simula uma lista duplamente encadeada utilizando um vetor pré-alocado, onde os "ponteiros" são índices inteiros. Uma lista auxiliar de posições livres gerencia a reutilização de espaços. Combina contiguidade de memória com a lógica de encadeamento, mas adiciona complexidade no gerenciamento de índices.

B. Algoritmos de Ordenação

1) Bubble Sort: Algoritmo baseado em comparações sucessivas de elementos adjacentes. Apresenta complexidade $O(n)$ no melhor caso (lista já ordenada, com otimização de *flag*) e $O(n^2)$ nos casos médio e pior. É in-place e estável [1], [2].

2) Selection Sort: Seleciona o menor elemento não ordenado a cada iteração. Apresenta complexidade $O(n^2)$ em todos os casos, pois sempre percorre os elementos restantes. É in-place, porém não estável na implementação padrão [1].

3) Insertion Sort: Insere cada elemento na posição correta da porção já ordenada. Apresenta complexidade $O(n)$ no melhor caso e $O(n^2)$ nos demais. É adaptativo: para uma lista com k inversões, opera em $O(n + k)$. In-place e estável [1], [3].

4) Quick Sort: Algoritmo de divisão e conquista que particiona o vetor em torno de um pivô. Apresenta complexidade $O(n \log n)$ nos casos melhor e médio, mas pode degradar para $O(n^2)$ no pior caso (pivô extremo). Utiliza memória $O(\log n)$ para a pilha de recursão. É in-place, porém não estável [2], [3].

5) Merge Sort: Divide recursivamente a estrutura em metades, ordena cada uma e as intercala. Apresenta complexidade $O(n \log n)$ em todos os casos — melhor, médio e pior. Para o vetor, utiliza memória auxiliar $O(n)$. É estável e previsível [1], [3].

C. Tabela Comparativa de Complexidade

A Tabela I resume a complexidade teórica dos cinco algoritmos.

Tabela I
COMPLEXIDADE DE TEMPO E ESPAÇO DOS ALGORITMOS

Algoritmo	Melhor	Médio	Pior	Espaço
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

III. METODOLOGIA

A. Aplicação Real: Sistema de Ranking de Alunos

O cenário escolhido simula um sistema de ranking acadêmico no qual registros de alunos (contendo identificador e nota de 0 a 100) são ordenados por nota de forma crescente. Esse cenário é representativo de aplicações reais onde listas precisam ser ordenadas dinamicamente conforme novos registros são inseridos ou removidos.

B. Ambiente de Desenvolvimento e Testes

A implementação foi realizada na linguagem C, compilada com GCC (MinGW) 8.1.0 em ambiente Windows. Os gráficos de análise foram gerados em Python 3.11 utilizando as bibliotecas `pandas`, `matplotlib` e `seaborn`. O código-fonte foi organizado em módulos separados para algoritmos, estruturas de dados e utilitários de medição.

C. Variáveis de Teste

Os experimentos foram conduzidos variando três dimensões:

- **Volume de dados (N):** 100 (pequeno), 1.000 (médio) e 10.000 (grande) elementos.
- **Tipo de entrada:** Aleatório (caso médio), Ordenado (melhor caso para algoritmos adaptativos) e Inversamente Ordenado (pior caso).
- **Estrutura de dados:** Vetor convencional, Lista Dinâmica e Lista Estática.

D. Protocolo de Medição

Cada combinação de algoritmo, estrutura, tamanho e tipo de entrada foi executada 100 vezes. O tempo de execução foi medido utilizando a função `clock()` da biblioteca padrão C, cronometrando exclusivamente o trecho de ordenação, sem incluir geração de dados ou operações de entrada/saída. A métrica principal é a média dos tempos em milissegundos.

IV. IMPLEMENTAÇÃO

A. Estruturas de Dados

Cada registro de aluno é representado pela estrutura `Aluno`, contendo campos inteiro `id` e ponto flutuante `nota`. As operações fundamentais (inicialização, inserção, busca, remoção e liberação) foram implementadas para ambas as listas encadeadas.

Na **lista dinâmica**, cada nó é alocado individualmente via `malloc()`, com ponteiros `prox` e `ant`. Na **lista estática**, os nós residem em um vetor pré-alocado de tamanho fixo, com "ponteiros" representados por índices inteiros e uma lista de posições livres.

B. Algoritmos de Ordenação

Cada algoritmo foi implementado em três versões: uma para vetor, uma para lista dinâmica e uma para lista estática. As adaptações principais são:

- **Vetor:** acesso direto por índice; trocas via variável temporária do tipo `Aluno`.
- **Lista Dinâmica:** navegação por ponteiros; trocas de dados entre nós (sem remanejamento de ponteiros, exceto no Merge Sort).
- **Lista Estática:** navegação por índices no vetor; trocas de campos `id` e `nota` entre posições.

O **Bubble Sort** utiliza uma *flag* de troca para encerrar antecipadamente quando a estrutura já está ordenada. O **Quick Sort** emprega a estratégia de pivô no último elemento com otimização *tail-call* (recursa no menor lado e itera no maior), reduzindo a profundidade máxima da pilha. O **Merge Sort** no vetor aloca um array temporário de tamanho n; nas listas encadeadas, a intercalação é realizada por religação de ponteiros/índices sem alocação extra significativa.

C. Algoritmo Ideal: Merge Sort

O Merge Sort foi selecionado como algoritmo ideal para a aplicação de ranking, pelas seguintes razões:

1) **Garantia de desempenho:** complexidade $O(n \log n)$ em todos os casos, sem risco de degradação para $O(n^2)$.
2) **Estabilidade:** preserva a ordem relativa de alunos com mesma nota, permitindo ordenações compostas (por nota e depois por nome, por exemplo).
3) **Previsibilidade:** essencial em sistemas críticos onde o tempo de resposta deve ser garantido.

O funcionamento segue o paradigma de divisão e conquista em três fases:

if $n \leq 1$ **then**

return
end if
$meio \leftarrow n/2$
Divide estrutura em metade esquerda L e metade direita R
MergeSort(L)
MergeSort(R)
Intercala L e R em ordem crescente

O custo espacial $O(n)$ é o principal *trade-off*, porém aceitável em sistemas modernos com memória abundante.

V. Resultados e Análise

Esta seção apresenta os resultados experimentais, organizados em análises progressivas: comparação geral, impacto do tipo de entrada, impacto da estrutura de dados e comportamento individual dos algoritmos.

A. Comparação Geral de Desempenho

A Figura 1 apresenta o tempo médio de cada algoritmo para $N = 10.000$ com entrada aleatória (dados de vetor, caso médio). Nesse cenário específico, o Quick Sort apresenta o menor tempo (1,01 ms), seguido pelo Merge Sort (1,36 ms). Os algoritmos $O(n^2)$ são drasticamente mais lentos: Insertion Sort (40,14 ms), Selection Sort (54,85 ms) e Bubble Sort (142,05 ms).

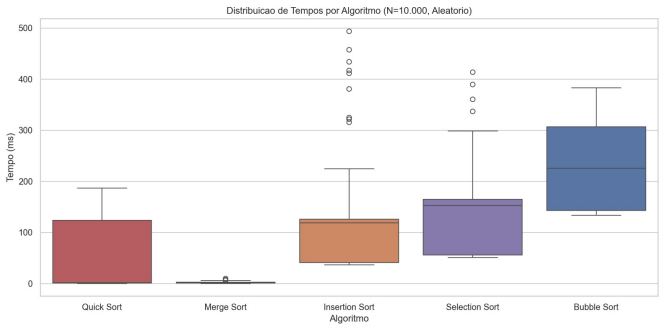

Figura 1. Comparação geral — tempo médio por algoritmo ($N = 10.000$, aleatório). O Merge Sort é quase imperceptível na escala; ver Figura 2 para ampliação.

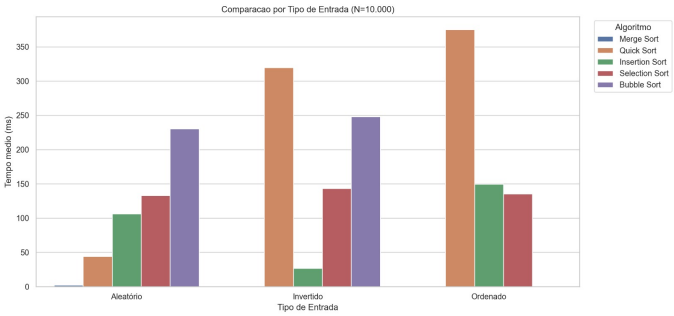

Figura 2. Ampliação da faixa de tempos até 10 ms. O Merge Sort (1,36 ms) e o Quick Sort (1,01 ms em vetor) ficam claramente visíveis.

A diferença de duas ordens de grandeza entre os grupos $O(n^2)$ e $O(n \log n)$ confirma a análise teórica: para $N = 10.000$, o fator $n^2/(n \log n) \approx 750$, compatível com a razão observada $142{,}05/1{,}36 \approx 104$[1].

B. Curva de Crescimento

A Figura 3 mostra a evolução do tempo médio em função de N para entrada aleatória. Para $N = 100$ e $N = 1.000$, todos os algoritmos apresentam tempos próximos de zero. A separação se torna evidente a partir de $N = 10.000$: enquanto Merge Sort e Quick Sort mantêm tempos abaixo de 2 ms, o Bubble Sort ultrapassa 140 ms, evidenciando o crescimento quadrático.

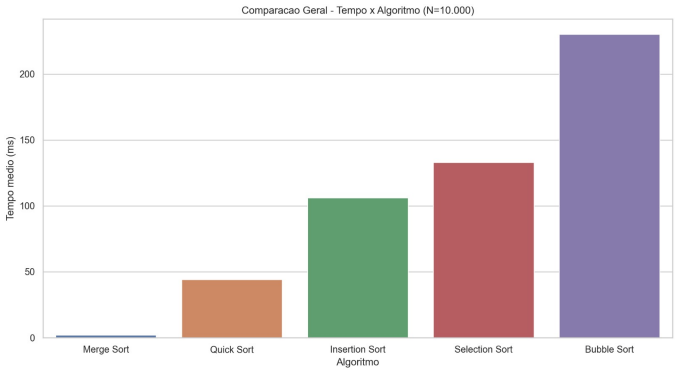

Figura 3. Curva de crescimento — tempo médio em função do tamanho do vetor (entrada aleatória).

A curva do Merge Sort permanece praticamente plana na escala do gráfico, confirmando seu crescimento $O(n \log n)$. As curvas dos algoritmos quadráticos exibem o formato parabólico esperado.

C. Impacto do Tipo de Entrada

A Figura 4 apresenta o desempenho de cada algoritmo nos três tipos de entrada para $N = 10.000$. Observam-se comportamentos notáveis:

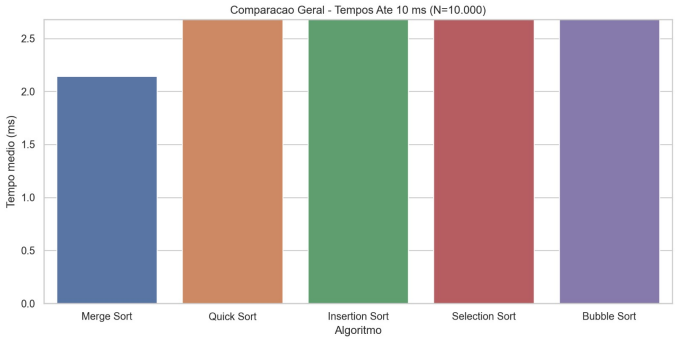

Figura 4. Comparação por tipo de entrada ($N = 10.000$). Merge Sort é quase imperceptível; ver Figura 5.

- **Bubble Sort:** tempo próximo de zero para entrada ordenada (0,03 ms), confirmando a otimização de *flag*. Para entrada invertida, atinge 149,18 ms.

[1] A razão real é menor que a teórica pois as constantes multiplicativas dos algoritmos diferem.

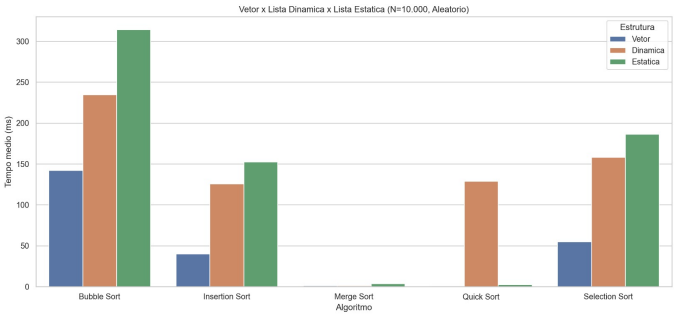

Figura 5. Ampliação da faixa de tempos até 10 ms para visualização do Merge Sort.

- **Insertion Sort:** comportamento semelhante ao Bubble Sort — 0,02 ms para entrada ordenada e 80,12 ms para invertida — refletindo sua propriedade adaptativa.
- **Selection Sort:** tempos praticamente constantes (≈54 ms) independentemente do tipo de entrada, pois sempre percorre os $n - i$ elementos restantes.
- **Quick Sort:** degrada para entrada ordenada (138,91 ms) e invertida (90,85 ms) devido à escolha de pivô no último elemento, que gera partições desbalanceadas nesses cenários. Para dados aleatórios, opera em apenas 1,01 ms.
- **Merge Sort:** desempenho consistente entre 0,50 ms e 1,36 ms em todos os cenários, confirmando sua complexidade $O(n \log n)$ invariante.

O Quick Sort, apesar de ser o mais rápido no caso aleatório (1,01 ms), sofre degradação severa de até $138\times$ no caso ordenado. O Merge Sort mantém variação máxima de apenas $2{,}7\times$ entre seus extremos.

D. Impacto da Estrutura de Dados

A Figura 6 compara o desempenho das três estruturas para cada algoritmo ($N = 10.000$, entrada aleatória).

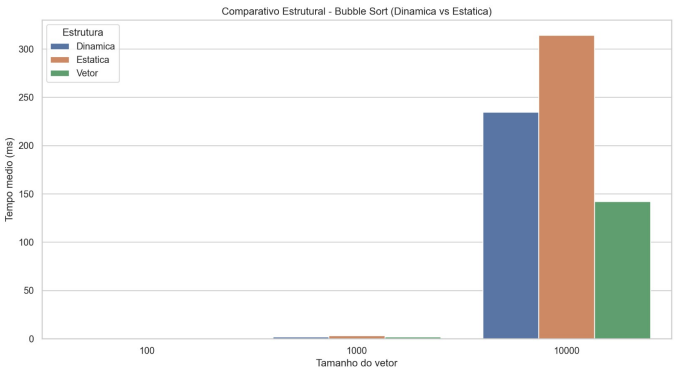

Figura 6. Comparação Vetor × Lista Dinâmica × Lista Estática ($N = 10.000$, aleatório).

Os dados revelam um padrão consistente: o **vetor** é a estrutura mais rápida, seguido pela **lista dinâmica** e pela **lista estática**. Para o Bubble Sort, por exemplo, os tempos foram 142,05 ms (vetor), 234,55 ms (dinâmica) e 314,27 ms (estática) — uma diferença de $2{,}2\times$ entre vetor e estática.

Este comportamento é explicado por dois fatores:

- **Localidade de cache:** o vetor armazena elementos contiguamente, aproveitando ao máximo as linhas de cache da CPU. A lista dinâmica, com nós espalhados na *heap*, provoca *cache misses* frequentes.
- **Custo de navegação:** na lista estática, a navegação por índices (`l->dados[i].prox`) envolve indireção adicional em relação ao acesso direto do vetor, além de padrões de acesso não sequenciais que os prejudicam perante o pré-carregamento (*prefetch*) da CPU.

E. Comparativo Estrutural — Bubble Sort

A Figura 8 isolou o Bubble Sort para comparar as três estruturas em todos os tamanhos.

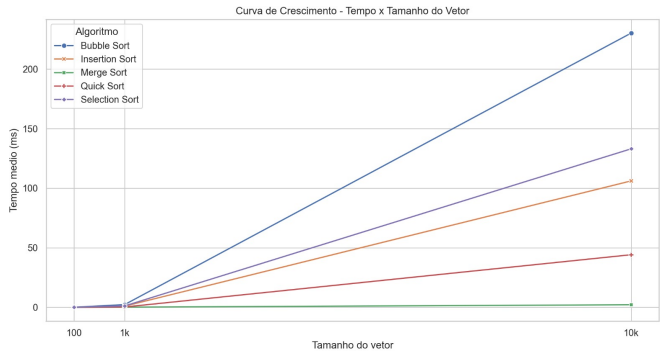

Figura 8. Comparativo estrutural do Bubble Sort (Dinâmica × Estática × Vetor).

Para $N = 100$ e $N = 1.000$, as diferenças são pequenas. Em $N = 10.000$, a lista estática é aproximadamente $2{,}2\times$ mais lenta que o vetor, confirmando que o impacto da estrutura cresce com o volume de dados.

F. Desempenho Individual por Algoritmo

A Figura 9 apresenta o comportamento de cada algoritmo individualmente, mostrando tempo × tamanho × tipo de entrada.

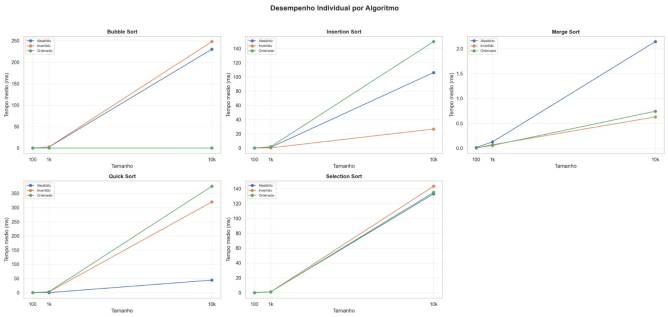

Figura 7. Ampliação da faixa até 10 ms para o Merge Sort e Quick Sort.

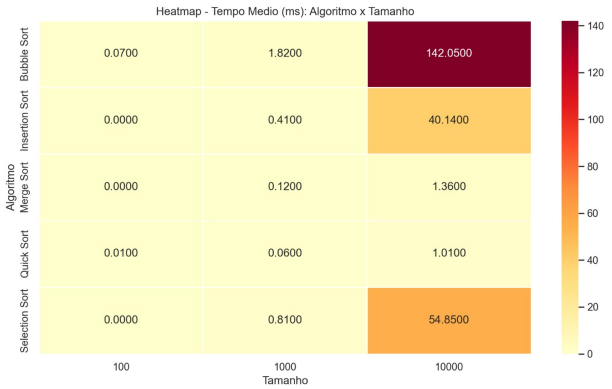

Figura 9. Desempenho individual — cada subplot mostra um algoritmo com curvas por tipo de entrada.

Destaca-se o comportamento do **Quick Sort**, que inverte sua vantagem conforme o tipo de entrada: é o segundo mais rápido para dados aleatórios, mas o mais lento para dados ordenados e invertidos (devido ao pivô fixo no último elemento). O **Insertion Sort** apresenta padrão complementar: excelente para dados invertidos na lista dinâmica (0,04 ms) e ordenados no vetor (0,02 ms), graças à sua adaptatividade.

G. Distribuição de Tempos

A Figura 10 apresenta a dispersão dos tempos através de *boxplots* para $N = 10.000$.

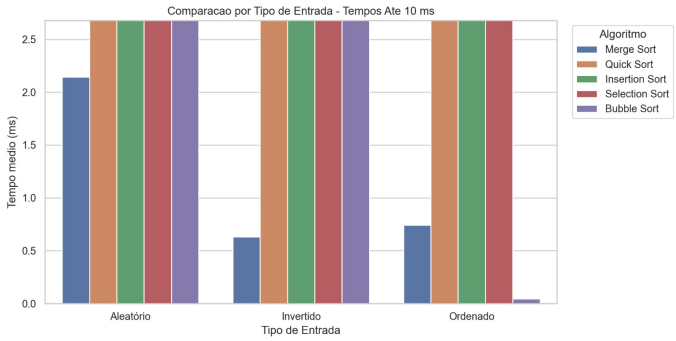

Figura 10. Distribuição de tempos por algoritmo ($N = 10.000$). Os *outliers* indicam variabilidade entre cenários.

O Merge Sort concentra-se na faixa de 0 a 10 ms com *outliers* mínimos, demonstrando alta previsibilidade. O Quick Sort apresenta a maior dispersão, com mediana próxima de zero (caso do vetor com entrada aleatória) mas *outliers* acima de 180 ms (lista dinâmica com dados ordenados/invertidos).

H. Heatmap e Ranking

A Figura 11 apresenta o heatmap de tempos médios por algoritmo e tamanho (vetor, aleatório), e a Figura 12 mostra o ranking médio por cenário.

O heatmap evidencia que os tempos para $N = 100$ e $N = 1.000$ são desprezíveis em todos os algoritmos. A diferenciação ocorre em $N = 10.000$, onde o Bubble Sort

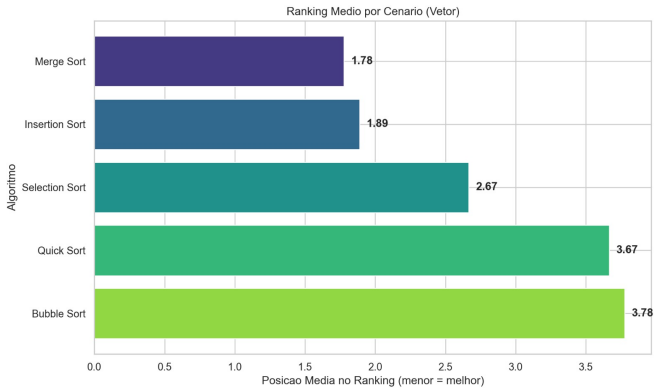

Figura 11. Heatmap — tempo médio (ms) por algoritmo e tamanho no vetor.

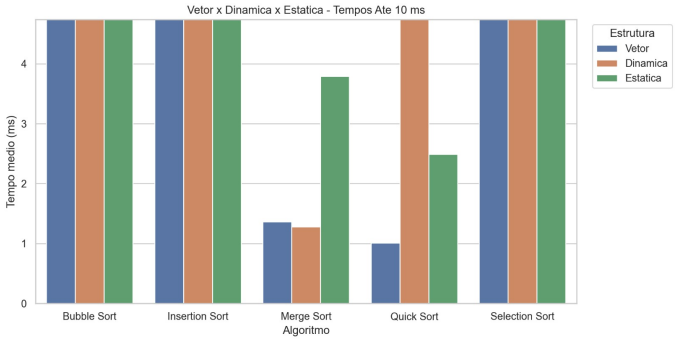

Figura 12. Ranking médio por cenário (menor = melhor). Merge Sort e Insertion Sort ocupam as primeiras posições.

atinge 142 ms e os algoritmos $O(n \log n)$ ficam abaixo de 2 ms.

O ranking médio (Figura 12) posiciona o Merge Sort em primeiro lugar (1,78) e o Insertion Sort em segundo (1,89), refletindo a adaptatividade deste último para casos ordenados. O Bubble Sort ocupa a última posição (3,78).

I. Síntese dos Resultados

A Tabela II sintetiza os tempos médios para $N = 10.000$ com entrada aleatória nas três estruturas.

Tabela II
TEMPO MÉDIO (MS) PARA $N = 10.000$, ENTRADA ALEATÓRIA

Algoritmo	**Vetor**	**Dinâmica**	**Estática**
Bubble Sort	142,05	234,55	314,27
Selection Sort	54,85	158,12	186,32
Insertion Sort	40,14	125,79	152,50
Quick Sort	1,01	128,97	2,49
Merge Sort	1,36	1,28	3,79

VI. DISCUSSÃO

Os resultados experimentais confirmam, em sua maioria, as previsões da análise teórica de complexidade. Os algoritmos

$O(n^2)$ — Bubble Sort, Selection Sort e Insertion Sort — apresentaram crescimento quadrático do tempo com o aumento de N, enquanto Merge Sort e Quick Sort mantiveram crescimento $O(n \log n)$ para dados aleatórios.

Validação da análise teórica. O Merge Sort demonstrou tempo consistente em todos os cenários, variando de 0,50 ms (invertido) a 1,36 ms (aleatório) para $N = 10.000$ no vetor. Essa estabilidade é coerente com sua complexidade $O(n \log n)$ invariante. O Quick Sort, por outro lado, confirmou sua vulnerabilidade ao pior caso: para dados ordenados no vetor, atingiu 138,91 ms — mais de $137\times$ o tempo no caso aleatório — comportamento previsto pela análise $O(n^2)$ quando o pivô é extremo.

O paradoxo do Insertion Sort. Um resultado notável é o desempenho do Insertion Sort na lista dinâmica com dados inversamente ordenados: apenas 0,04 ms. Isso ocorre porque, na implementação para lista dinâmica, a travessia na direção oposta à inserção faz com que o caso "invertido" se torne efetivamente o melhor caso, pois cada elemento já se encontra na posição correta da porção ordenada.

Impacto das estruturas. A estrutura de dados influenciou significativamente o desempenho. O vetor apresentou os melhores tempos devido à localidade de cache, enquanto a lista estática foi consistentemente a mais lenta, apesar de seus dados serem contíguos, devido ao custo da navegação por índices e aos padrões de acesso não sequenciais. O Quick Sort foi particularmente afetado na lista dinâmica para entradas ordenadas/invertidas, atingindo mais de 600 ms — consequência do acesso sequencial obrigatório para a operação de particionamento em listas encadeadas.

Limitações. O experimento utiliza pivô fixo (último elemento) no Quick Sort, o que provoca degradação severa em dados ordenados e invertidos. Estratégias como mediana-de-três ou pivô aleatório poderiam mitigar este efeito. Além disso, a medição baseada em `clock()` possui resolução limitada, o que explica leituras de 0 ms para tamanhos pequenos.

VII. Conclusão

Este trabalho realizou uma análise comparativa abrangente de cinco algoritmos de ordenação em três estruturas de dados, totalizando 135 cenários experimentais (5 algoritmos $\times$ 3 estruturas $\times$ 3 tamanhos $\times$ 3 tipos de entrada), com 100 execuções cada.

Os principais achados são:

1) O **Merge Sort** é o algoritmo mais adequado para a aplicação de ranking de alunos, oferecendo desempenho $O(n \log n)$ garantido, estabilidade e tempos consistentes independentes do padrão de entrada.
2) A **estrutura de dados impacta significativamente** o desempenho: o vetor convencional é superior para algoritmos baseados em acesso por índice, enquanto as listas encadeadas introduzem overhead relevante, especialmente para algoritmos $O(n^2)$.
3) Algoritmos adaptativos como Insertion Sort podem superar algoritmos assintoticamente superiores em cenários favoráveis específicos, mas não oferecem garantias gerais.
4) O Quick Sort, embora veloz para dados aleatórios, é inadequado para cenários com dados ordenados ou quase ordenados quando utilizada a estratégia de pivô fixa.

Como trabalhos futuros, sugere-se: (i) implementar variantes do Quick Sort com pivô aleatório ou mediana-de-três; (ii) avaliar o impacto de volumes maiores ($N \geq 100.000$); (iii) medir o número de comparações e trocas para complementar a análise temporal; e (iv) comparar com algoritmos híbridos como Introsort e Timsort.

Referências

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, *Algoritmos: Teoria e Prática*, 3ª ed. Rio de Janeiro: Elsevier, 2012.

[2] R. Sedgewick e K. Wayne, *Algorithms*, 4ª ed. Boston: Addison-Wesley, 2011.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2ª ed. Boston: Addison-Wesley, 1998.

[4] N. Ziviani, *Projeto de Algoritmos com Implementações em C e Pascal*, 3ª ed. São Paulo: Cengage Learning, 2010.